The "Context-First" Prompting Workflow (for Cursor & Replit)

Most people fail with AI coding because they give the AI too much "noise" or too little "context." Here is the 3-step loop I will teach you.

Step 1: The Context Setup (The "@" Rule)

Before typing a single word of instructions, we define the scope using Cursor's `@` symbols.

- **`@Files`:** Only tag the specific components we are editing.
- **`@Docs`:** Attach the official Supabase or Next.js docs so the AI uses the latest API versions, not outdated data.
- **`@Codebase`:** Use this sparingly for "big picture" questions.

Step 2: The "Chain of Thought" Prompt

Instead of saying "Build a login page," we use a structural prompt:

Role: Senior Next.js Developer. **Task:** Create a responsive login form using Tailwind CSS. **Context:** Use the `@auth.ts` file as a reference for the backend logic. **Constraint:** Do not use external libraries for the form; use native React state. **Output:** Step-by-step implementation including error handling.

Step 3: The Validation Loop

Once the AI generates the code, we don't just "Accept" it. We run a verification check:

1. **Terminal Check:** Use `@Terminal` to feed errors back into Cursor.
2. **Logic Review:** "Why did you choose this Hook over `useEffect`?" (This is where the mentoring happens).
3. **Refactor:** Ask the AI to "Check for edge cases or security vulnerabilities in the code you just wrote."

Step 3: The Validation Loop

Once the AI generates the code, we don't just "Accept" it. We run a verification check:

1. **Terminal Check:** Use `@Terminal` to feed errors back into Cursor.
2. **Logic Review:** "Why did you choose this Hook over `useEffect`?" (This is where the mentoring happens).
3. **Refactor:** Ask the AI to "Check for edge cases or security vulnerabilities in the code you just wrote."

Part 3: The ".cursorrules" Secret Sauce

To make your client's IDE "smarter" than everyone else's, I'll help you set up a `.cursorrules` file in your root directory. This is a hidden file that tells Cursor:

- "Always use TypeScript, never JavaScript."
- "Use functional components with arrow functions."
- "Always include comments explaining the 'Why' behind complex logic."
- "Prioritize mobile-first design with Tailwind."

This ensures the AI learns *your* preferences, so you don't have to repeat yourself in every chat.